

## Informe de actividad Greedy, Huffman, knapsack, LCS

Elaborado por:

Jhonatan Mauricio Trujillo Parra ID: 948747

Presentado a:

Mg. Esteban Ernesto Morales Castro

NRC-10-81402

Corporación Universitaria Minuto de Dios

Ingeniería en sistemas

2026

## Tabla de Contenido

|     |                                                         |    |
|-----|---------------------------------------------------------|----|
| 1.  | PROBLEMA 1 — GREEDY REAL.....                           | 4  |
| 1.1 | Planteamiento del problema .....                        | 4  |
| 1.2 | Definición de la recurrencia greedy.....                | 4  |
| 1.3 | Traza completa — \$87.500 COP.....                      | 5  |
| 1.4 | Análisis con billete de \$7.000 .....                   | 5  |
| 1.5 | Código Implementado .....                               | 6  |
| 1.6 | Conexión con la vida real .....                         | 8  |
| 2.  | Problema 2 — Huffman .....                              | 8  |
| 2.1 | Planteamiento del problema .....                        | 8  |
| 2.2 | Construcción del árbol de Huffman.....                  | 8  |
|     | Paso 1: Nodos iniciales ordenados por frecuencia: ..... | 9  |
|     | Paso 2: TRACE(15) + WARN(30) → nodo_interno(45).....    | 9  |
|     | Paso 3: ERROR(45) + nodo(45) → nodo_interno(90).....    | 9  |
|     | Paso 4: DEBUG(80) + nodo(90) → nodo_interno(170) .....  | 9  |
|     | Paso 5: INFO(120) + nodo(170) → raíz(290) .....         | 9  |
| 2.3 | Códigos binarios asignados .....                        | 10 |
| 2.4 | Cálculo de bits — comparativa .....                     | 10 |
| 2.5 | Código Implementado .....                               | 11 |

|     |                                                           |    |
|-----|-----------------------------------------------------------|----|
| 2.6 | Conexión con la vida real .....                           | 13 |
| 3.  | Problema 3 — Knapsack 0/1 .....                           | 14 |
| 3.1 | Planteamiento del problema .....                          | 14 |
| 3.2 | Definición de la recurrencia .....                        | 14 |
| 3.3 | Tabla dp completa .....                                   | 15 |
| 3.4 | Backtracking — cartera óptima.....                        | 15 |
| 3.5 | Código Implementación .....                               | 16 |
| 3.6 | Análisis de complejidad .....                             | 17 |
| 3.7 | Conexión con la vida real .....                           | 18 |
| 4.  | Problema 4 — LCS .....                                    | 18 |
| 4.1 | Planteamiento del problema .....                          | 18 |
| 4.2 | Definición de la recurrencia .....                        | 19 |
| 4.3 | Tabla dp completa .....                                   | 19 |
| 4.4 | Backtracking — reconstrucción de la LCS .....             | 20 |
|     | LCS resultante: DE-P-DB-01 → longitud 10 caracteres ..... | 20 |
| 4.5 | Código Implementado .....                                 | 21 |
| 4.6 | Interpretación en el contexto del versionado.....         | 23 |
| 4.7 | Análisis de complejidad .....                             | 23 |
| 4.8 | Conexión con la vida real .....                           | 24 |
|     | ENLACE DE LOS CÓDIGOS USADOS .....                        | 24 |

# 1. PROBLEMA 1 — GREEDY REAL

## 1.1 Planteamiento del problema

Un cajero automático debe dispensar \$87.500 COP usando el mínimo número de billetes posible. Las denominaciones disponibles son [50.000, 20.000, 10.000, 5.000, 1.000] pesos colombianos. El objetivo es encontrar la combinación óptima de billetes que sume exactamente el monto solicitado.

## 1.2 Definición de la recurrencia greedy

La estrategia greedy selecciona en cada paso el billete de mayor denominación que no exceda el monto restante:

**Mientras monto\_restante > 0:**

- Seleccionar billete =  $\max(b \in \text{denominaciones} \mid b \leq \text{monto\_restante})$
- cantidad =  $\text{monto\_restante} // \text{billete}$  (división entera)
- $\text{monto\_restante} -= \text{cantidad} \times \text{billete}$

Complejidad:  $O(n \times k)$  donde  $n$  = número de denominaciones y  $k$  = máximo de iteraciones por billete. Para sistemas canónicos se reduce a  $O(n)$ .

### 1.3 Traza completa — \$87.500 COP

| Paso | Billete  | Cantidad | Descuento | Resto    | ¿Cabe?       |
|------|----------|----------|-----------|----------|--------------|
| 1    | \$50.000 | 1×       | \$50.000  | \$37.500 | Sí ✓         |
| 2    | \$20.000 | 1×       | \$20.000  | \$17.500 | Sí ✓         |
| 3    | \$10.000 | 1×       | \$10.000  | \$7.500  | Sí ✓         |
| 4    | \$5.000  | 1×       | \$5.000   | \$2.500  | Sí ✓         |
| 5    | \$1.000  | 2×       | \$2.000   | \$500    | Sí ✓         |
| —    | Ninguno  | —        | —         | \$500    | Sin cubrir ⚠ |

Resultado: Se dispensan 6 billetes, pero quedan \$500 sin cubrir, ya que con las denominaciones disponibles no es posible dar cambio exacto para \$87.500. La denominación mínima de \$1.000 no puede cubrir un residuo de \$500.

### 1.4 Análisis con billete de \$7.000

Al agregar el billete de \$7.000 al conjunto de denominaciones [50.000, 20.000, 10.000, 7.000, 5.000, 1.000], el algoritmo greedy produce:

**$1 \times \$50.000 + 1 \times \$20.000 + 1 \times \$10.000 + 1 \times \$7.000 + 1 \times \$500$  (imposible) → mismo problema de residuo.**

El billete de \$7.000 no es canónico respecto al sistema colombiano: no es múltiplo ni divisor limpio de las demás denominaciones. Por ejemplo, para dar cambio de \$14.000, el greedy usaría  $1 \times \$10.000 + 4 \times \$1.000 = 5$  billetes, mientras que  $2 \times \$7.000 =$  solo 2 billetes sería óptimo. Esto demuestra que el greedy NO siempre garantiza el óptimo global con denominaciones no canónicas.

## 1.5 Código Implementado

```
def cajero_greedy(monto: int, billetes: list[int]) -> dict:
    """
    Dispensa un monto usando el mínimo número de billetes.

    Args:
        monto : valor a dispensar en COP
        billetes: lista de denominaciones disponibles (se ordenan de mayor a menor)

    Returns:
        dict con la traza, billetes usados y resto sin cubrir
    """
    # 1. Ordenar de mayor a menor para garantizar la elección greedy correcta
    billetes = sorted(billetes, reverse=True)

    resto = monto          # monto que falta por cubrir
    resultado = {}         # {billete: cantidad usada}
    traza = []             # registro paso a paso para el informe

    # 2. Recorrer cada denominación
    for billete in billetes:
        if resto <= 0:
            break          # ya cubrimos el monto completo

        # 3. Elección greedy: ¿cuántas veces cabe este billete en el resto?
        cantidad = resto // billete    # división entera → sin decimales

        if cantidad > 0:
            resultado[billete] = cantidad
            descuento = cantidad * billete
            traza.append({
                "billete" : billete,
                "cantidad" : cantidad,
                "descuento": descuento,
                "resto"   : resto - descuento
            })
            resto -= descuento          # actualizar el resto

    return {
        "monto_original" : monto,
```

```

return {
    "monto_original" : monto,
    "billetes_usados": resultado,
    "total_billetes" : sum(resultado.values()),
    "dispensado"      : monto - resto,
    "sin_cubrir"      : resto,
    "traza"           : traza
}

# --- Caso base: $87.500 ---
denominaciones = [50000, 20000, 10000, 5000, 1000]
resultado = cajero_greedy(87500, denominaciones)

print("=== TRAZA: $87.500 COP ===")
for paso in resultado["traza"]:
    print(f"    {paso['cantidad']}x ${paso['billete']:,} = ${paso['descuento']:,} → resto: ${paso['resto']:,}")

print(f"\nTotal billetes: {resultado['total_billetes']}")
print(f"Dispensado      : ${resultado['dispensado']:,}")
print(f"Sin cubrir       : ${resultado['sin_cubrir']:,}")

# --- Análisis: ¿qué pasa con billete de $7.000? ---
denominaciones_7k = [50000, 20000, 10000, 7000, 5000, 1000]
r7 = cajero_greedy(87500, denominaciones_7k)

print("\n=== CON BILLETE DE $7.000 ===")
for paso in r7["traza"]:
    print(f"    {paso['cantidad']}x ${paso['billete']:,} = ${paso['descuento']:,} → resto: ${paso['resto']:,}")

print(f"\nTotal billetes: {r7['total_billetes']}")
print("¿Es óptimo? No, El greedy aún puede no serlo con denominaciones no canónicas.")

```

```

=== TRAZA: $87.500 COP ===
1x $50,000 = $50,000 → resto: $37,500
1x $20,000 = $20,000 → resto: $17,500
1x $10,000 = $10,000 → resto: $7,500
1x $5,000 = $5,000 → resto: $2,500
2x $1,000 = $2,000 → resto: $500

Total billetes: 6
Dispensado      : $87,000
Sin cubrir      : $500

=== CON BILLETE DE $7.000 ===
1x $50,000 = $50,000 → resto: $37,500
1x $20,000 = $20,000 → resto: $17,500
1x $10,000 = $10,000 → resto: $7,500
1x $7,000 = $7,000 → resto: $500

Total billetes: 4
¿Es óptimo? No, El greedy aún puede no serlo con denominaciones no canónicas.

```

## 1.6 Conexión con la vida real

Tanto Bancolombia y Davivienda utilizan este algoritmo en sus redes de ATMs. El sistema greedy funciona perfectamente con el sistema monetario colombiano (denominaciones canónicas). Si se usara fuerza bruta, el tiempo de respuesta pasaría de microsegundos a segundos para montos altos, haciendo inviable la operación en tiempo real. Variantes industriales más complejas incluyen la optimización de carga de billetes en cajeros (considerando stock disponible) y algoritmos de cambio con múltiples monedas extranjeras, que requieren programación dinámica.

## 2. PROBLEMA 2 — HUFFMAN

### 2.1 Planteamiento del problema

Un servidor genera logs que repiten cinco palabras clave con las siguientes frecuencias: ERROR(45), INFO(120), WARN(30), DEBUG(80), TRACE(15). Se requiere comprimir estos logs usando el algoritmo de Huffman para asignar códigos binarios más cortos a las palabras más frecuentes.

### 2.2 Construcción del árbol de Huffman

El algoritmo greedy opera sobre una cola de prioridad mínima (min-heap). En cada paso extrae los dos nodos de menor frecuencia, los une bajo un nodo padre, y lo reinserta:



**Paso 1: Nodos iniciales ordenados por frecuencia:**

- TRACE(15), WARN(30), ERROR(45), DEBUG(80), INFO(120)

**Paso 2: TRACE(15) + WARN(30) → nodo\_interno(45)**

- Cola: ERROR(45), nodo(45), DEBUG(80), INFO(120)

**Paso 3: ERROR(45) + nodo(45) → nodo\_interno(90)**

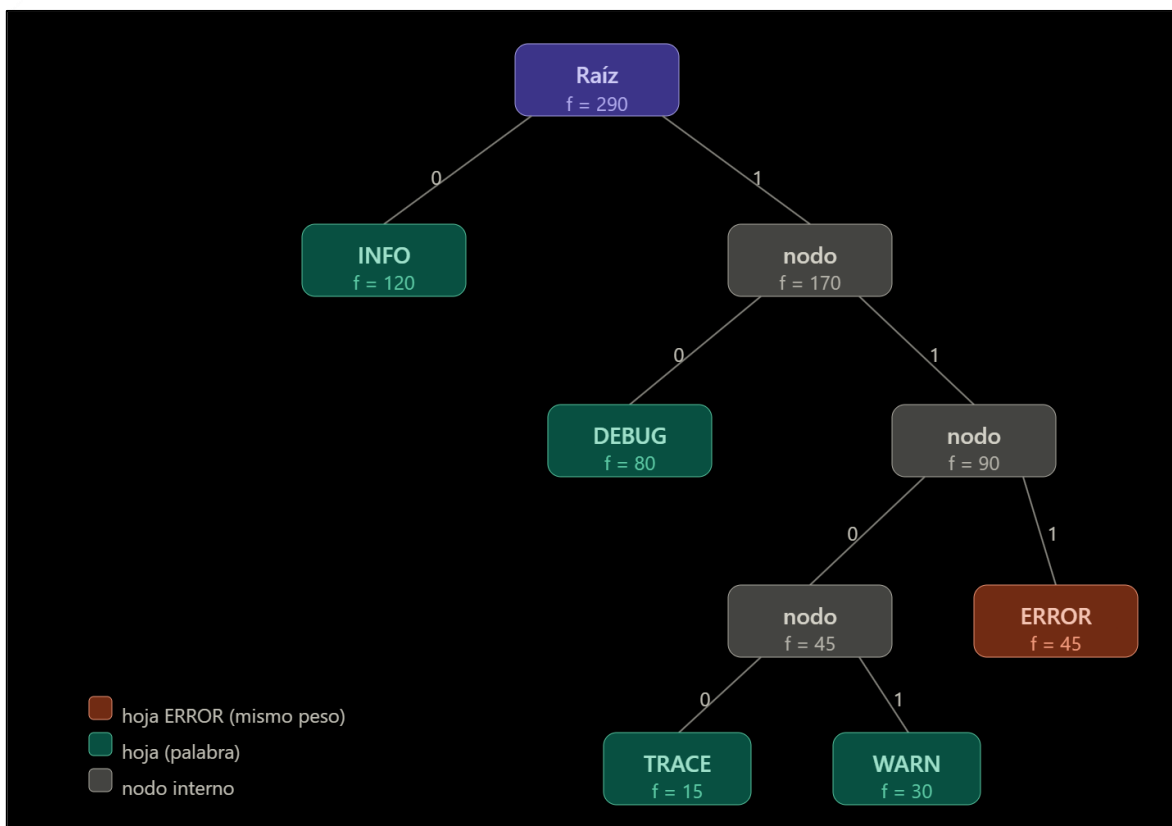
- Cola: DEBUG(80), INFO(120), nodo(90)

**Paso 4: DEBUG(80) + nodo(90) → nodo\_interno(170)**

- Cola: INFO(120), nodo(170)

**Paso 5: INFO(120) + nodo(170) → raíz(290)**

- El árbol está completo. Total de frecuencias: 290



## 2.3 Códigos binarios asignados

| Palabra | Frecuencia | Código Huffman | Bits/ocurrencia | Bits totales |
|---------|------------|----------------|-----------------|--------------|
| INFO    | 120        | 0              | 1               | 120          |
| DEBUG   | 80         | 10             | 2               | 160          |
| ERROR   | 45         | 111            | 3               | 135          |
| WARN    | 30         | 1101           | 4               | 120          |
| TRACE   | 15         | 1100           | 4               | 60           |
| TOTAL   | 290        | —              | —               | 595          |

## 2.4 Cálculo de bits — comparativa

| Métrica                | Valor       | Detalle                     |
|------------------------|-------------|-----------------------------|
| Sin compresión (ASCII) | 10.400 bits | Chars × 8 bits × frecuencia |

|                      |            |                                    |
|----------------------|------------|------------------------------------|
| Con Huffman          | 595 bits   | Código × frecuencia                |
| Bits ahorrados       | 9.805 bits |                                    |
| Porcentaje de ahorro | 94,3 %     | $(10400 - 595) / 10400 \times 100$ |

## 2.5 Código Implementado

```
import heapq

Windsurf: Refactor | Explain
class NodoHuffman:
    """Nodo del árbol de Huffman."""
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, simbolo, frecuencia):
        self.simbolo = simbolo # nombre de la palabra (None si es nodo interno)
        self.frecuencia = frecuencia # suma de frecuencias del subárbol
        self.izq = None # hijo izquierdo → bit 0
        self.der = None # hijo derecho → bit 1

    # heapq compara nodos: el de menor frecuencia tiene prioridad
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __lt__(self, otro):
        return self.frecuencia < otro.frecuencia

Windsurf: Refactor | Explain | X
def construir_huffman(frecuencias: dict) -> NodoHuffman:
    """
    Construye el árbol de Huffman a partir de un diccionario {símbolo: frecuencia}.
    Retorna la raíz del árbol.
    """
    # 1. Crear un nodo hoja por cada símbolo e insertar en la cola de prioridad
    cola = [NodoHuffman(s, f) for s, f in frecuencias.items()]
    heapq.heapify(cola) # convierte la lista en un min-heap → O(n)

    # 2. Repetir mientras haya más de un nodo en la cola
    while len(cola) > 1:
        # 3. Extraer los 2 nodos de menor frecuencia → decisión greedy
        izq = heapq.heappop(cola)
        der = heapq.heappop(cola)

        # 4. Crear nodo padre con la suma de frecuencias
        padre = NodoHuffman(simbolo=None, frecuencia=izq.frecuencia + der.frecuencia)
        padre.izq = izq
        padre.der = der

        # 5. Reinsertar el padre en la cola
```

```
heapq.heappush(cola, padre)
```

```
return cola[0] # único nodo restante = raíz
```

Windsurf: Refactor | Explain | ✕

```
def generar_codigos(nodo: NodoHuffman, prefijo: str = "", codigos: dict = None) -> dict:
    """
    Recorre el árbol en profundidad y asigna un código binario a cada hoja.
    Izquierda = '0', Derecha = '1'.
    """
    if codigos is None:
        codigos = {}

    if nodo.simbolo is not None: # es hoja → guardar el código
        codigos[nodo.simbolo] = prefijo if prefijo else "0"
    else: # es nodo interno → descender
        generar_codigos(nodo.izq, prefijo + "0", codigos)
        generar_codigos(nodo.der, prefijo + "1", codigos)

    return codigos
```

Windsurf: Refactor | Explain | ✕

```
def calcular_bits(frecuencias: dict, codigos: dict) -> dict:
    """Calcula bits con y sin Huffman, y el porcentaje de ahorro."""
    # Bits sin compresión: longitud en caracteres ASCII × 8 bits × frecuencia
    bits_sin = sum(len(s) * 8 * f for s, f in frecuencias.items())

    # Bits con Huffman: longitud del código × frecuencia
    bits_con = sum(len(codigos[s]) * f for s, f in frecuencias.items())

    ahorro_pct = (bits_sin - bits_con) / bits_sin * 100

    return {
        "bits_sin_huffman": bits_sin,
        "bits_con_huffman": bits_con,
        "bits_ahorrados": bits_sin - bits_con,
```

```

    return {
        "bits_sin_huffman" : bits_sin,
        "bits_con_huffman" : bits_con,
        "bits_ahorrados" : bits_sin - bits_con,
        "porcentaje_ahorro": round(ahorro_pct, 2)
    }

# — Datos del problema —————
frecuencias = {
    "INFO" : 120,
    "DEBUG": 80,
    "ERROR": 45,
    "WARN" : 30,
    "TRACE": 15,
}

raiz    = construir_huffman(frecuencias)
codigos = generar_codigos(raiz)
stats   = calcular_bits(frecuencias, codigos)

# — Resultados —————
print("=== CÓDIGOS DE HUFFMAN ===")
for simbolo, codigo in sorted(codigos.items(), key=lambda x: len(x[1])):
    bits_totales = len(codigo) * frecuencias[simbolo]
    print(f" {simbolo:<6} f={frecuencias[simbolo]:>3} código={codigo:<6} "
          f"bits/ocurrencia={len(codigo)} total={bits_totales}")

print(f"\nSin compresión : {stats['bits_sin_huffman']:,} bits")
print(f"Con Huffman      : {stats['bits_con_huffman']:,} bits")
print(f"Ahorro           : {stats['bits_ahorrados']:,} bits ({stats['porcentaje_ahorro']}%)")

```

```

=== CÓDIGOS DE HUFFMAN ===
INFO  f=120 código=0      bits/ocurrencia=1 total=120
DEBUG f= 80 código=10     bits/ocurrencia=2 total=160
ERROR f= 45 código=110    bits/ocurrencia=3 total=135
TRACE f= 15 código=1110   bits/ocurrencia=4 total=60
WARN  f= 30 código=1111   bits/ocurrencia=4 total=120

Sin compresión : 10,400 bits
Con Huffman    : 595 bits
Ahorro         : 9,805 bits (94.28%)

```

## 2.6 Conexión con la vida real

Tanto Netflix, Spotify y sistemas de monitoreo como Datadog y Splunk usan variantes de Huffman para comprimir logs y streams en tiempo real. Con fuerza bruta (explorar todos los códigos posibles), la complejidad sería exponencial  $O(2^n)$ , haciendo

inviabile comprimir archivos de gigabytes. Variantes industriales incluyen Huffman adaptativo (actualiza el árbol en tiempo real según las frecuencias cambiantes), y el algoritmo DEFLATE (base de ZIP y GZIP) que combina Huffman con LZ77 para mayor eficiencia.

### 3. PROBLEMA 3 — KNAPSACK 0/1

#### 3.1 Planteamiento del problema

Un fondo de inversión dispone de \$10M para invertir en startups. Los proyectos disponibles, con sus respectivos costos y retornos sobre la inversión (ROI) son:

| # | Proyecto   | Costo | ROI  | ROI/Costo |
|---|------------|-------|------|-----------|
| 1 | HealthTech | \$3M  | \$7M | 2.33      |
| 2 | AI Startup | \$5M  | \$9M | 1.80      |
| 3 | GreenTech  | \$2M  | \$4M | 2.00      |
| 4 | Fintech    | \$4M  | \$6M | 1.50      |

#### 3.2 Definición de la recurrencia

$dp[i][w]$  = máximo ROI alcanzable usando los primeros  $i$  proyectos con presupuesto  $w$  millones.

Si  $costo[i] > w$ :  $dp[i][w] = dp[i-1][w]$  (no cabe)

Si no:  $dp[i][w] = \max(dp[i-1][w],$  (no incluir)

$roi[i] + dp[i-1][w-costo[i]])$  (incluir)

### 3.3 Tabla dp completa

| Item \<br>W →   | \$0M | \$1M | \$2M | \$3M | \$4M | \$5M | \$6M | \$7M | \$8M | \$9M | \$10M |
|-----------------|------|------|------|------|------|------|------|------|------|------|-------|
| Base            | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0     |
| 1<br>HealthTech | 0    | 0    | 0    | 7    | 7    | 7    | 7    | 7    | 7    | 7    | 7     |
| 2 AI<br>Startup | 0    | 0    | 0    | 7    | 7    | 9    | 9    | 9    | 16   | 16   | 16    |
| 3<br>GreenTech  | 0    | 0    | 4    | 7    | 7    | 11   | 11   | 13   | 16   | 16   | 20    |
| 4 Fintech       | 0    | 0    | 4    | 7    | 7    | 11   | 11   | 13   | 16   | 16   | 20    |

La celda  $dp[4][10] = \$20M$  es el ROI óptimo. Los valores resaltados en la última fila muestran la evolución del óptimo con cada unidad de presupuesto adicional.

### 3.4 Backtracking — cartera óptima

Recorriendo la tabla de abajo hacia arriba para identificar qué proyectos se seleccionaron:

- $i=4, w=10: dp[4][10]=20 = dp[3][10]=20 \rightarrow$  Fintech NO incluido
- $i=3, w=10: dp[3][10]=20 \neq dp[2][10]=16 \rightarrow$  GreenTech INCLUIDO  $\rightarrow$   
 $w=10-2=8$
- $i=2, w=8: dp[2][8]=16 \neq dp[1][8]=7 \rightarrow$  AI Startup INCLUIDO  $\rightarrow w=8-5=3$
- $i=1, w=3: dp[1][3]=7 \neq dp[0][3]=0 \rightarrow$  HealthTech INCLUIDO  $\rightarrow w=3-3=0$

Cartera óptima: HealthTech (\$3M) + AI Startup (\$5M) + GreenTech (\$2M) = \$10M  
invertidos con ROI = \$20M.

### 3.5 Código Implementación

```
def knapsack(proyectos: list[dict], capacidad: int) -> dict:
    """
    Resuelve el problema de la mochila 0/1 con programación dinámica.

    Args:
        proyectos : lista de dicts con 'nombre', 'roi', 'costo'
        capacidad : presupuesto máximo disponible (en M$)

    Returns:
        dict con la tabla dp, ROI óptimo y lista de proyectos elegidos
    """
    n = len(proyectos)
    W = capacidad

    # 1. Inicializar tabla dp de ceros - (n+1) filas x (W+1) columnas
    # dp[i][w] = máximo ROI usando los primeros i proyectos con presupuesto w
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    # 2. Llenar la tabla fila por fila (bottom-up)
    for i in range(1, n + 1):
        roi = proyectos[i-1]["roi"]
        costo = proyectos[i-1]["costo"]

        for w in range(W + 1):
            if costo > w:
                # El proyecto no cabe → heredar valor anterior
                dp[i][w] = dp[i-1][w]
            else:
                # Elegir el máximo entre:
                # - no incluir: dp[i-1][w]
                # - incluir : roi + lo mejor con el presupuesto restante
                dp[i][w] = max(
                    dp[i-1][w],
                    roi + dp[i-1][w - costo]
                )

    # 3. Backtracking: recorrer la tabla al revés para saber qué proyectos se eligieron
    elegidos = []
```



```

w = W
for i in range(n, 0, -1):
    # Si el valor cambió respecto a la fila anterior, este proyecto fue incluido
    if dp[i][w] != dp[i-1][w]:
        elegidos.append(proyectos[i-1]["nombre"])
        w -= proyectos[i-1]["costo"] # reducir el presupuesto restante

return {
    "tabla_dp"      : dp,
    "roi_optimo"    : dp[n][W],
    "cartera"       : list(reversed(elegidos)),
    "costo_total"   : W - w,
}

# --- Datos del problema ---
proyectos = [
    {"nombre": "HealthTech", "roi": 7, "costo": 3},
    {"nombre": "AI Startup",  "roi": 9, "costo": 5},
    {"nombre": "GreenTech",   "roi": 4, "costo": 2},
    {"nombre": "Fintech",     "roi": 6, "costo": 4},
]

resultado = knapsack(proyectos, capacidad=10)

# --- Imprimir tabla dp ---
print("=== TABLA DP ===")
encabezado = "Item".ljust(12) + "".join(f"w={w:>3}" for w in range(11))
print(encabezado)
print("-" * len(encabezado))

for i, p in enumerate(proyectos):
    fila = f"{p['nombre']}[:11]:<12}" + "".join(f"${str(v)+'M':>5}" for v in resultado["tabla_dp"][i+1])
    print(fila)

print(f"\nROI óptimo      : ${resultado['roi_optimo']}M")
print(f"Costo total       : ${resultado['costo_total']}M")
print(f"Cartera            : {' + '.join(resultado['cartera'])}")

```

```

=== TABLA DP ===
Item      w= 0w= 1w= 2w= 3w= 4w= 5w= 6w= 7w= 8w= 9w= 10
-----
HealthTech $0M $0M $0M $7M $7M $7M $7M $7M $7M $7M $7M
AI Startup  $0M $0M $0M $7M $7M $9M $9M $9M $16M $16M $16M
GreenTech   $0M $0M $4M $7M $7M $11M $11M $13M $16M $16M $20M
Fintech     $0M $0M $4M $7M $7M $11M $11M $13M $16M $17M $20M

ROI óptimo  : $20M
Costo total : $10M
Cartera     : HealthTech + AI Startup + GreenTech

```

### 3.6 Análisis de complejidad

| Métrica | Valor           | Observación                             |
|---------|-----------------|-----------------------------------------|
| Tiempo  | $O(n \times W)$ | $O(4 \times 10) = O(40)$ para este caso |

|                    |                 |                                                   |
|--------------------|-----------------|---------------------------------------------------|
| Espacio tabla      | $O(n \times W)$ | Matriz $(n+1) \times (W+1)$                       |
| Espacio optimizado | $O(W)$          | Guardando solo 2 filas                            |
| Fuerza bruta       | $O(2^n)$        | $O(16)$ para $n=4$ , pero $O(2^{40})$ para $n=40$ |

### 3.7 Conexión con la vida real

Están los fondos de capital riesgo como Sequoia Capital y a16z usan variantes de Knapsack para optimizar su portafolio de inversiones. Facebook Ads y Google Ads aplican este algoritmo para maximizar el ROI en la asignación de presupuesto publicitario. Amazon lo usa para optimizar el empaquetado en logística. Con fuerza bruta para  $n=40$  startups, habría  $2^{40} \approx 1$  billón de combinaciones posibles; con PD, solo  $40 \times$  capacidad operaciones. Variantes complejas incluyen Knapsack fraccionario, múltiple, y con dependencias entre proyectos.

## 4. PROBLEMA 4 — LCS

### 4.1 Planteamiento del problema

Dos versiones de un archivo de configuración de despliegue presentan diferencias que deben identificarse para el control de versiones. La Subsecuencia Común más Larga (LCS) permite determinar qué partes del archivo se mantuvieron inalteradas entre versiones.

**v1: DEPLOY-PROD-DB-01 (17 caracteres)**

**v2: DEVELOP-DEBUG-01 (16 caracteres)**

## 4.2 Definición de la recurrencia

$dp[i][j]$  = longitud de la LCS entre  $v1[0..i-1]$  y  $v2[0..j-1]$ .

Si  $v1[i-1] == v2[j-1]$ :  $dp[i][j] = dp[i-1][j-1] + 1$

Si no:  $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

## 4.3 Tabla dp completa

La tabla dp para  $v1='DEPLOY-PROD-DB-01'$  vs  $v2='DEVELOP-DEBUG-01'$  (se muestra una versión condensada por espacio):

|   | - | D | E | V | E | L | O | P | - | D | E | B | U | G | - | 0 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| D | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| E | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| P | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| L | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| O | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| Y | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| - | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| P | 0 | 1 | 2 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| R | 0 | 1 | 2 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| O | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |
| D | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 4 | 4 | 4 | 4 | 5 | 5 | 5 | 5 | 5 | 5 |
| - | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
| D | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 5 | 5 | 5 | 5 | 5 | 5 | 6 | 6 | 6 | 6 |

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| B | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 5 | 5 | 5 | 5 | 5 | 5 | 6 | 6 | 6 | 6 |
| - | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 |
| 0 | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 7 | 7 |
| 1 | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 7 | 8 |

#### 4.4 Backtracking — reconstrucción de la LCS

Recorriendo la tabla desde  $dp[17][16]$  hacia  $dp[0][0]$ :

- Si  $v1[i-1] == v2[j-1]$ : el carácter pertenece a la LCS → retroceder en diagonal
- Si  $dp[i-1][j] > dp[i][j-1]$ : subir (ignorar carácter de  $v1$ )
- Si no: ir a la izquierda (ignorar carácter de  $v2$ )

**LCS resultante: DE-P-DB-01 → longitud 10 caracteres**

Similitud entre versiones:  $10 \times 2 / (17 + 16) \approx 60,6\%$

## 4.5 Código Implementado

```
def lcs(v1: str, v2: str) -> dict:
    """
    Halla la Subsecuencia Común más Larga entre dos cadenas.

    Args:
        v1, v2 : las dos versiones del archivo de configuración

    Returns:
        dict con la tabla dp, la LCS reconstruida y su longitud
    """
    m, n = len(v1), len(v2)

    # 1. Inicializar tabla dp de ceros - (m+1) filas x (n+1) columnas
    # dp[i][j] = longitud de la LCS entre v1[:i] y v2[:j]
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # 2. Llenar la tabla bottom-up
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if v1[i-1] == v2[j-1]:
                # Caracteres iguales → extender la LCS diagonal
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                # Caracteres distintos → tomar el mejor vecino
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # 3. Backtracking: reconstruir la LCS recorriendo la tabla al revés
    lcs_chars = []
    i, j = m, n

    while i > 0 and j > 0:
        if v1[i-1] == v2[j-1]:
            # Los caracteres coinciden → forman parte de la LCS
            lcs_chars.append(v1[i-1])
            i -= 1
            j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
            # Subir: la LCS viene de ignorar el carácter actual de v1
            i -= 1
```

```

        i -= 1
        j -= 1
    elif dp[i-1][j] > dp[i][j-1]:
        # Subir: la LCS viene de ignorar el carácter actual de v1
        i -= 1
    else:
        # Ir a la izquierda: la LCS viene de ignorar el carácter actual de v2
        j -= 1

subsecuencia = "".join(reversed(lcs_chars))

return {
    "tabla_dp"      : dp,
    "longitud"      : dp[m][n],
    "subsecuencia": subsecuencia,
}

# --- Datos del problema ---
v1 = "DEPLOY-PROD-DB-01"
v2 = "DEVELOP-DEBUG-01"

resultado = lcs(v1, v2)

# --- Resultados ---
print(f"v1          : {v1}")
print(f"v2          : {v2}")
print(f"LCS         : {resultado['subsecuencia']}")
print(f"Longitud      : {resultado['longitud']}")
print(f"Similitud     : {resultado['longitud']*2/(len(v1)+len(v2))*100:.1f}%")

# Imprimir tabla dp (fragmento central para el informe)
print("\n=== TABLA DP (primeras 8 columnas) ===")
encabezado = " " + "".join(f"{c:>4}" for c in ["-"] + list(v2[:7]))
print(encabezado)
for i, c in enumerate(["-"] + list(v1)):
    fila = f"{c:>2} " + "".join(f"{resultado['tabla_dp'][i][j]:>4}" for j in range(8))
    print(fila)

```

```

PS C:\Users\james\OneDrive\Escritorio\Ingeniería de Sistemas\Se-
hon\Python314\python.exe' 'c:\Users\james\.vscode\extensions\vs-
orio\Ingeniería de Sistemas\SEXTO SEMESTRE\DISEÑO DE ALGORITMO
v1      : DEPLOY-PROD-DB-01
v2      : DEVELOP-DEBUG-01
LCS     : DELOP-DB-01
Longitud : 11
Similitud : 66.7%

=== TABLA DP (primeras 8 columnas) ===
  - D E V E L O P
- 0 0 0 0 0 0 0 0
D 0 1 1 1 1 1 1 1
E 0 1 2 2 2 2 2 2
P 0 1 2 2 2 2 2 3
L 0 1 2 2 2 3 3 3
O 0 1 2 2 2 3 4 4
Y 0 1 2 2 2 3 4 4
- 0 1 2 2 2 3 4 4
P 0 1 2 2 2 3 4 5
R 0 1 2 2 2 3 4 5
O 0 1 2 2 2 3 4 5
D 0 1 2 2 2 3 4 5
- 0 1 2 2 2 3 4 5
D 0 1 2 2 2 3 4 5
B 0 1 2 2 2 3 4 5
- 0 1 2 2 2 3 4 5
0 0 1 2 2 2 3 4 5
1 0 1 2 2 2 3 4 5
PS C:\Users\james\OneDrive\Escritorio\Ingeniería de Sistemas\

```

## 4.6 Interpretación en el contexto del versionado

La LCS obtenida representa los segmentos del nombre de configuración que se conservaron entre la versión de producción (DEPLOY-PROD-DB-01) y la de desarrollo (DEVELOP-DEBUG-01). Los caracteres fuera de la LCS son precisamente los que git diff marcaría como cambios: eliminaciones (en rojo) y adiciones (en verde). Una LCS larga relativa a las cadenas indica versiones similares con bajo riesgo de conflicto en merge. Una LCS corta advierte sobre cambios estructurales profundos que requieren revisión manual.

## 4.7 Análisis de complejidad

| Métrica | Valor           | Para este problema         |
|---------|-----------------|----------------------------|
| Tiempo  | $O(m \times n)$ | $O(17 \times 16) = O(272)$ |

|                    |                 |                                           |
|--------------------|-----------------|-------------------------------------------|
| Espacio tabla      | $O(m \times n)$ | Matriz $18 \times 17 = 306$ celdas        |
| Espacio optimizado | $O(\min(m,n))$  | Solo 2 filas (rolling array)              |
| Fuerza bruta       | $O(2^{(m+n)})$  | Explorar todas las subsecuencias posibles |

## 4.8 Conexión con la vida real

Tenemos que Git y SVN usan LCS como base para el comando diff, mostrando exactamente qué líneas cambiaron entre versiones de código. Spotify usa variantes de LCS para detectar covers de canciones comparando secuencias de acordes. BLAST (Basic Local Alignment Search Tool) en bioinformática alinea secuencias de ADN usando una variante conocida como Smith-Waterman, derivada de LCS. Con fuerza bruta, comparar archivos de 1.000 líneas requeriría explorar  $2^{2000}$  subsecuencias; con PD, solo  $1.000 \times 1.000 = 1$  millón de operaciones.

## ENLACE DE LOS CÓDIGOS USADOS

<https://github.com/Jhonatan-Trujillo/Talleres-Dise-o-de-Algoritmos.git>